LARAVEL PORTFOLIO

Project Documentation

A personal portfolio built with Laravel 12, Inertia.js and React, with a login-protected page for managing projects.

Ahrone B. Fabro · November 2026

Contents

1 Overview
2 Technology
3 Installing and running the project
4 Using the admin pages
5 How the project is organised
6 Routes
7 Project data and validation
8 Login and security
9 Testing
10 Putting the site online
11 Known limitations

1. Overview

The site has two parts: a public portfolio that anyone can view, and admin pages where the owner manages the projects shown on it.

Public portfolio

- Sections: About, Skills, Projects, Education and Contact ("Let's talk").
- Projects are loaded from the database and can be filtered by category: web, mobile, design or school.
- Light and dark themes, with the visitor's choice remembered.
- Scroll animations that respect the "reduce motion" setting, plus a responsive layout and mobile menu.
- Contact form that autosaves a draft in the browser, so a refresh doesn't lose a half-written message.

Admin pages

- Add, edit and delete projects, including a cover image upload with preview.
- Server-side validation with clear error messages. After a failed save, the page moves to the first problem.
- Delete confirmation, and success messages after each change.
- Login required. There is no public sign-up.

2. Technology

Layer	Tool	Used for
Back end	Laravel 12 (PHP 8.2+)	Routing, validation, login, file storage
Bridge	Inertia.js 2	Lets Laravel controllers render React pages without a separate API
Front end	React 19	Pages and components
Styling	Tailwind CSS 4	Layout, themes and animation classes
Build	Vite 6	Bundles the JavaScript and CSS
Database	SQLite	Projects, users and sessions
Tests	PHPUnit	Automated feature tests

3. Installing and running the project

You need PHP 8.2 or newer, Composer and Node.js. Run these in the project folder:

```
composer install
npm install
cp .env.example .env
php artisan key:generate
php artisan migrate --seed
php artisan storage:link
php artisan admin:create you@example.com
npm run build
php artisan serve
```

Command	What it does
`migrate --seed`	Creates the database tables and loads the six starting projects
`storage:link`	Makes uploaded images publicly viewable
`admin:create`	Creates the admin login and asks you to choose a password
`npm run build`	Builds the front end. Use `npm run dev` instead while developing, and keep it running
`serve`	Starts the site at http://127.0.0.1:8000

4. Using the admin pages

Opening the admin pages

1. Start the site with `php artisan serve` and leave that terminal running.
2. Go to http://127.0.0.1:8000/projects in your browser. The public site has no link to this page, so you need to enter the address.
3. Log in with your email and password.

Managing projects

- **Add:** click "+ Add project", fill in the form and upload a cover image.
- **Edit:** click "Edit" on a project. Leave the image field empty to keep the current image.
- **Delete:** click "Delete" and confirm. The project and its image are permanently removed.
- **Finish:** click "Log out" at the top of the page. To stop the site, press Ctrl+C in the terminal.

Changing the password

Run the same command with the same email, then type the new password when asked. You don't need the old password. The characters won't appear as you type, and there is no second prompt to confirm it.

```
php artisan admin:create you@example.com
```

Using a different email creates a second account, which also has full access.

5. How the project is organised

Part	Location
Routes	`routes/web.php`
Admin account command	`routes/console.php`
Project add, edit, delete and validation	`app/Http/Controllers/ProjectController.php`
Public page data	`app/Http/Controllers/PortfolioController.php`
Login and logout	`app/Http/Controllers/LoginController.php`
Project model	`app/Models/Project.php`
Projects table	`database/migrations/2026_10_21_121541_create_projects_table.php`
Starting data and images	`database/seeders/ProjectSeeder.php`, `database/seeders/images/`
Public page	`resources/js/Pages/Portfolio.jsx`
Login page	`resources/js/Pages/Login.jsx`
Admin pages	`resources/js/Pages/Projects/` (Index, Create, Edit)
Admin page frame	`resources/js/Layouts/AdminLayout.jsx`
Shared components	`resources/js/Components/`
Tests	`tests/Feature/`

6. Routes

Method	Address	Purpose	Login needed
GET	`/`	Public portfolio	No
GET	`/login`	Login form	No
POST	`/login`	Log in (limited to 5 attempts a minute)	No
POST	`/logout`	Log out	Yes
GET	`/projects`	List projects	Yes
GET	`/projects/create`	New project form	Yes
POST	`/projects`	Save a new project	Yes
GET	`/projects/{id}/edit`	Edit project form	Yes
PUT/PATCH	`/projects/{id}`	Save changes	Yes
DELETE	`/projects/{id}`	Delete a project and its image	Yes

7. Project data and validation

Each project is one row in the `projects` table. The server checks these rules on every save.

Field	Holds	Rules
`title`	Card heading	Required, 3-60 characters, unique
`categories`	Filter categories	At least one of web, mobile, design, school, with no repeats
`description`	Short summary	Required, 20-300 characters
`tech_stack`	Tools, separated by commas	Required, up to 120 characters
`project_url`	Link to the project	Optional, full http or https address, up to 255 characters
`image`	Cover image	JPG, PNG or WEBP, up to 2 MB. Required when adding, optional when editing
`image_alt`	Image description for screen readers	Required, 10-200 characters
`date_completed`	Completion date	Required, valid date, not in the future
`display_order`	Position on the page, lowest first	Required, whole number from 1 to 99

8. Login and security

- All project routes require a login. Visitors who aren't logged in are sent to `/login`.
- Accounts can only be created from the command line, and passwords must be at least 8 characters. Passwords are stored hashed.
- Login attempts are limited to 5 a minute, and the session is renewed at login and ended at logout.
- Uploads are limited to JPG, PNG and WEBP images, and project links must use http or https.
- Any account in the database has full access, because there are no separate roles.

9. Testing

```
php artisan test
```

The latest run passed all 21 tests (108 assertions).

Test file	Tests	Covers
`tests/Feature/ProjectCrudTest.php`	12	Pages load, add, edit, delete, validation errors, image replacement and project order
`tests/Feature/LoginTest.php`	8	Blocked access, correct and wrong passwords, attempt limit, logout and the admin command
`tests/Unit/ExampleTest.php`	1	Laravel's default example test

The 12 form tests, login, logout, move to the first error and skip link also passed in Chrome. Link previews need a public URL, so they are untested.

10. Putting the site online

1. Replace the placeholder project dates (see section 11).
2. Set the production values below in the server's `.env` file. With debug left on, visitors can see error details.
3. Run `php artisan migrate --seed`, `php artisan storage:link` and `npm run build` on the server.
4. Run `php artisan admin:create` with your email and a strong password. The server has its own database, so your local login won't carry over.

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://your-domain.example
```

11. Known limitations

The contact form doesn't deliver messages. It shows "Your message was submitted", but nothing is sent or stored, so the owner never receives these messages. Until this is fixed, visitors can use the email address shown beside the form. A working form needs a Laravel endpoint, a mail service, and spam protection.

Project dates are placeholders. The completion dates on the project cards aren't the real ones. Update them on the admin pages, and in `ProjectSeeder.php` if the database will be seeded again.

Smaller issues

- Project titles are only unique when capitals match, so "fitpass" and "FitPass" can both exist.
- Repeating a tool, as in "Figma, Figma", shows it twice on the card.
- The filter buttons are written separately from the category list in the model, so a new category must be added in both places.
- If a delete fails, the confirmation dialog closes without showing an error.
- Some seed images are large (`thorz-garage.png` is about 700 KB). Converting them to WebP would speed up the projects section.